

S1000d AppLicability dEpended pRocessiNg mOdel - SALERNO

1st Theresia El Khoury
Nanomedicine Lab

Université Bourgogne Franche-Comté
Belfort, France
0000-0002-8383-4005
theresia.el-khoury@utbm.fr

2nd Georges Badr
TICKET Lab.

Antonine University
Baabda, Lebanon
0000-0001-7404-9569
georges.badr@ua.edu.lb

3rd Amir Hajjam El Hassani
Nanomedicine Lab

Université Bourgogne Franche-Comté
Belfort, France
0000-0002-8470-806X
amir.hajjam@utbm.fr

4th Stéphane N'Guyen Van Ky

Département de Recherche & Développement
STUDEC
Blagnac, France
0000-0003-3805-0305
sngvk@studec.fr

Abstract—The widespread adoption of Machine Learning and Artificial Intelligence across different domains can be attributed to the digitization of data over several decades, resulting in vast amounts of data, types and structures. Thus, data processing and preparation turns out to be a crucial stage. However, applying these techniques to S1000D standard-based data poses a challenge due to its complexity and the need to preserve logical information. This paper describes SALERNO, an S1000d AppLicability dEpended pRocessiNg mOdel. This python-based model analyzes and converts the XML S1000D-based files, into an easier data format that can be used in machine learning techniques while preserving the different logic and relationships in files. The model parses the files in the given folder, filters them, and extracts the required information to be saved in appropriate data frames and Excel sheets. Its main idea is to group the extracted information by applicability. In addition, it extracts the full text by replacing internal and external references, while maintaining the relationships between files, as well as the necessary requirements. Resulting files can be then saved in databases and used in different models. Documents in both English and French languages were tested, and special characters were decoded. Updates on the technical manuals were taken into consideration as well. The model was tested on different versions of the S1000D, and the results demonstrated its ability to effectively handle the applicability, requirements, references, and relationships across all files and on different levels.

Index Terms—Aeronautics, Big data, Data processing, Machine learning, S1000D

I. INTRODUCTION

In the aeronautics and aerospace fields, technical publications are the root element for all appliances. They store every detail from the description to the maintenance and all possible procedures and processes that can be applied to each piece of equipment of an aircraft. These documents are created based

on the S1000D, an 'International specification for technical publications using a 'Common Source Database' (CSBD) [1], as part of the S-Series of ILS specifications [2]. It is a standard for creating, managing, and using technical information in the form of technical manuals, maintenance manuals, and other types of technical documentation. Preserving these information, leads to very detailed and structured data, making their search more and more complex. Artificial Intelligence can be used to help retrieve specific information from Big Data, but specific data processing should be applied first. The purpose is to convert this structure to an easier, computer-comprehensible format. Unlike other formats, the S1000D is not only text, rather, information is highly structured and subject to specific rules. Extracting only the text will remove all standards and knowledge in the data. On the other hand, using the XML format documents will be very complicated, especially in real-time process.

This paper introduces SALERNO, a Python-based software, used to pre-process XML files written specifically in S1000D format. The logic behind it is to extract the main details from the files, and convert them to a more readable and simpler format while keeping the logic within the information. The resulted data can be used for multiple data mining, machine learning, and information retrieval tasks.

The paper is formed as follows. The first section lists the different softwares used to process S1000D documents. In the second section, the S1000D specifications are detailed. The main model is explained in the third section, with the resulting data format. Finally, examples and results are shown in the last section, followed by the conclusion.

II. RELATED WORKS

Authors in [3] converted the S1000D data standards into Sharable Content Object Reference Model (SCORM) standard [4]. The approach used involves creating SNS-IETP and PM-

This article is supported by STUDEC.

979-8-3503-1806-7/23/\$31.00 ©2023 IEEE

IETP models by modeling Interactive Electronic Technical Publication (IETP) using Standard Numbering System (SNS) and Publication Module (PM) dimensions. Additionally, the electronic training system data is modeled to generate the SCORMP model. The three models are then compared using various business scenarios, and the PM-IETP model is chosen for the conversion process into the SCORMP standard.

To address the challenges posed by traditional paper-based technology data storage systems, which are difficult to maintain and inefficient, an Interactive Electronic Technical Manual (IETM) has been designed based on the S1000D specification in [5]. A mapping relationship between XML and relational databases is established in line with the S1000D specification, and a method for converting between XML documents and relational databases is provided. To improve the ease of human-computer interaction, the study explores the use of speech recognition in the creation of the IETM platform, and the process of IETM creation is explained.

A very important subsystem in the IETM system is the authoring system. From different types of data modules, the process data module is responsible for organizing other data modules to create interactive modules within the S1000D data module. The design of an XML-based data process module editor is proposed in [6]. A process DM data model is proposed based on the S1000D standard, and graphical objects and classes are set up accordingly using a Model-View-Controller architecture.

On a more narrowed view, an EMU IETM based on the S1000D specification is found in [7]. Based on the practical operation and maintenance situation of high-speed trains in China, a general framework with key functions for building an EMU technical information platform is designed. The research emphasizes the maintenance requirement and proposes a cross-organizational technical information collaboration workflow, as well as a business integration model for the platform and other railway production systems.

To the authors' knowledge, no model that processes data to the desired format, with the specific requirements was found. SALERNO was built from scratch, to process S1000D documents in different versions, languages, and for different types of DMs. Complex XML documents are converted into python data frames, representing all characteristics, relationships, requirements, content, and applicability.

III. S1000D SPECIFICATIONS

A. Description

It was initially developed by the AeroSpace and Defence Industries Association of Europe (ASD) [8] to standardize common specifications across the services. Now it is owned by ASD, the Aerospace Industries Association of America (AIA) [9], and the ATA e-Business Program [10]. This XML-based norm has become the international specification for technical publications using a CSBD. Every technical documentation should follow this norm in order to be accepted and used in aeronautics, aerospace, and the military.

It's a very detailed and structured standard but relies on a

common idea that every piece of information, is created as an individual data item, called a data module (DM), structured with predefined XML elements and metadata. Content is thereby modularized with each DM, allowing its easy reuse with multiple outputs. DMs can have several types, regarding their objectives. The main types endorsed are :

- **Crew**, Contains info used only by the crew and operators,
- **Description**, Gives a full description of a specific behavior,
- **Procedural**, Lists all the steps required to perform a unique procedure,
- **Fault**, Captures and represents fault reporting, isolation, and correlation info,
- **Illustrated Parts Data**, Known as IPD, provides lists of illustrated breakdowns with their corresponding part,
- **Schedules**, Sets the maintenance time limits for appropriate equipment,
- **Checklist**, Exclusive for aeronautics, used when a procedure must present a checklist,
- **Process**, Provides the different interactive structures, which can sequence other DMs or steps within it.

Multiple versions of the specification are available, differing in the node tags used and some hierarchical details. Issues 3.0 and above differ in an evident way from issues 2.3 and below. Nevertheless, the main components of a DM, alongside the rules and logic behind this hierarchy are practically the same. This includes the relationships between DMs, and the applicability, which can be available for an entire DM, or for specific parts of it, on a very fine level of the hierarchy. Applicability allows for the identification of the specific context in which a data module or its components are considered to be valid. Its range of capabilities can vary significantly, ranging from a straightforward annotation embedded within a data module's content to a more comprehensive management of the applicability life cycle. It is a Boolean expression formed by attribute tags, which can accumulate infinitely, attending a very complex level for human interpretation.

B. Data Module Composition

Regardless of the type and version, all DM files can incorporate various elements, such as text, multimedia, and references to illustrations or to other data modules. It comprises two parts: an identification and status section that contains all the management information for that data module, and a content section that includes the information visible to the user, and differs based on the type. The unique identifier for each data module is a Data Module Code (DMC). The most important information chosen to be extracted from each data module are described below.

In the status node, a unique DMC is used as a set of values to represent each data module, alongside its title, type, version, issue type, number, and date, in addition to the applicability of the entire data module.

In the content section, the main part is chosen based on the type. For some actions, mainly for a procedure, some requirements may exist before starting and after finishing the

steps. When preliminary requirements exist, they are listed in the first part of the content, while close requirements are listed in the last part. Some sections of the content can have their own applicability, thereby related to specific applicability in the DM. In addition, internal reference IDs may be used to refer to a specific node in the file. Data modules can also be related by referencing their DMCs in the content.

All these details are taken into consideration, in order to have a proper logical interpretation of the data modules.

IV. MODEL

Our model is designed to tackle, not only the easy conversion issues but also the structuring issues found in the S1000D. Starting with special characters handling, and full-text extraction, including replacing all internal and external references. Then, it focuses on the identification of unique DMCs, references, requirements, and relationships between DMs, and most importantly, the applicability corresponding to each segment of text, nodes, and children. It also takes into consideration the updates of the technical manual, by adding new documents, updating existing ones, and deleting them when necessary. All these specifications are saved in different data frames during the process, then saved respectively in Excel files, and finally uploaded to the database.

From each XML file, the model extracts:

- Data Module Characteristics
 - the unique DMC and title
 - the type and version
 - the issue type, number and date
 - the applicability of the entire file, if present
- Data Module Content
 - the preliminary and close requirements with each of its applicability when available
 - the full text with each of its applicability
 - replace the internal and external reference IDs with the corresponding text

The model takes as a parameter the path of the repository in which all data is saved. Note that files other than XML can be found, and some XML files can be saved as text or HTML files. Thereby, the first important step is to convert these files into XML, remove all special characters, if exist, and save them as valid XML files in order to be parsed. All files in the folder are filtered and parsed for the characteristic extraction and saved in a data module data frame. The same files are then parsed again to extract the content, where this data frame is used to map the DMC reference in each content to the corresponding data module. Resulting data frames are saved in Excel sheets, and the database is then updated correspondingly.

A. Data Module Characteristics

This first phase is intended to gather all identification and status details about all the data modules and save them in a specific data module data frame. Fig. 1 shows that for each XML file, a data module instance is created, to extract all characteristics, including the DMC, type, version, and status.

As mentioned, for each version, some tags can differ, so after creating a data module instance, the appropriate tags and paths are set based on the document's version. Since not all types are taken into consideration, type filtering is added before continuing the process. For the constant updates of the technical manual, the issue type and number in this phase are extracted to determine if the document should be added, updated, or deleted. Available documents in the database are extracted to check for updates. A document is new, if not found in the extracted data, while it's considered updated if its issue number is incremented. The documents having an issue type set to 'deleted' are added to a single data frame, new and updated files are parsed, while unchanged ones are disregarded.

If a data module refers to another, this creates a relationship between both. In order to contextualize this, a list of all data module references is saved, if available, resulting in a reference data frame, listing all DMC referenced in each document.

In addition to these main details, some data modules may have applicability mentioned in the status section. The applicability is added to the applicability data frame, and to the data module instance and saved alongside the other attributes for each unique DMC in the data frame. The data module data frame, containing all basic information about each file, will be the reference for having a full text, forming the content.

B. Data Module Content

In the second phase, the content part of the document is parsed. For each XML file in the folder, the model creates a data module entity and check its type. The same filtering process of the status phase is used. Only files that have been added or updated, from the types defined, are processed. We save the applicabilities of each document, in a content applicability data frame. Then a recurrent function is called to extract the full text alongside its applicability ID for each node. The data frame saved in the first phase is used to match the applicability ID with its value, in order to save the full details in the content data frame.

For types having preliminary or close requirements, the appropriate recurrent function is called to extract the requirements, grouping them by applicability IDs. Two data frames result from this, a preliminary requirements data frame, and another for close requirements. In this case, not only node text is extracted but also attributes. For all text, or requirements that have no applicability ID, they inherit the applicability of the document, found in the status section, in case it exists.

1) *Text and applicability extraction:* When extracting the full text, the model takes into consideration several points:

- nodes may have no applicability,
- some nodes may have their own applicability, which automatically applies to their children nodes,
- different children of the same node may have different applicabilities, overriding their parents' applicability,

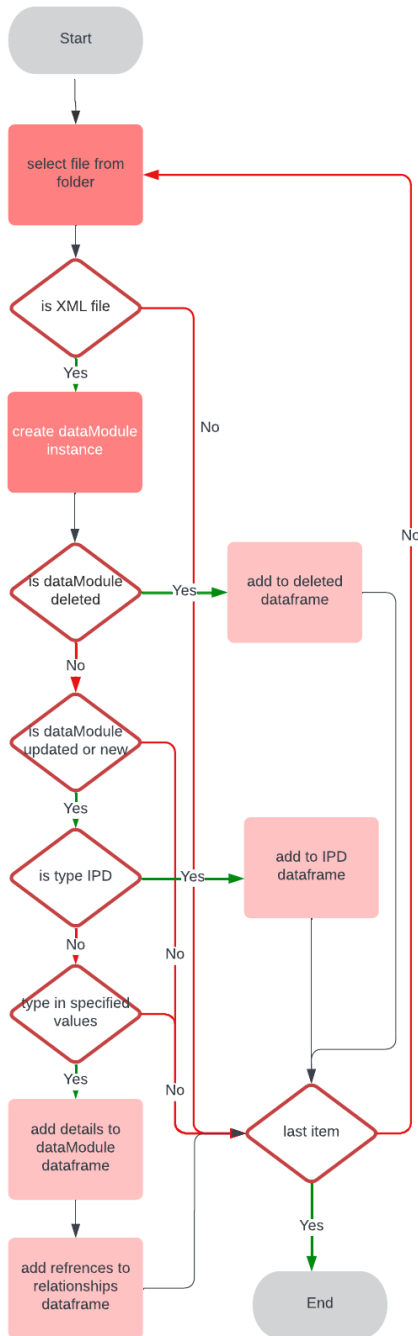


Fig. 1. Flowchart of the characteristics extraction phase

- text in nodes may be a simple string, concatenated with all children's text,
- text in nodes may be interrupted by an internal reference node that should be replaced by its corresponding text,
- text in nodes may be interrupted by an external reference node, which should be replaced by its title,
- text should be saved under its matching applicability.

This logic is applied to all the children nodes of the content.

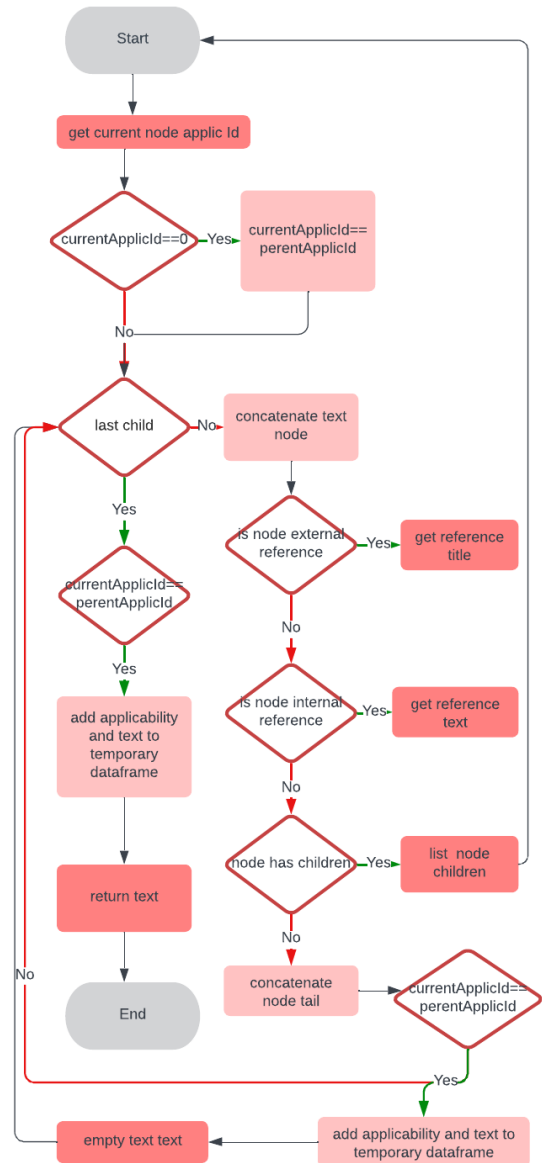


Fig. 2. Flowchart of the recurrent function used to replace and save each text with corresponding applicability

The flowchart in Fig. 2 shows the different stages of the function. The recurrent function takes as a parameter the node, and its parent applicability ID if available. The first node is the content itself. It checks if the current node has a different applicability ID than its parent's. Then, for each child node of the current one, the text is concatenated.

If the node is an external reference, its children are not parsed, but its title is concatenated with the text. If the title is a child of the reference, then this title is added, otherwise, the DMC of the reference is used to get the appropriate title from the data frame containing all data module information.

If the node is an internal reference, the reference ID is used to find the corresponding node in the document and concatenate

its text, and children's text to the current one.

If the node has children and is not a reference, then for all its children, the recurrent function is called with the current applicability, and so on.

If the node has no children, and has an applicability different than the parent, then the text is saved into the content data frame with the corresponding applicability. If not, the text is returned by the function and concatenated with the parent's text to be added under the right applicability. If no applicability is listed for a document, then all the text is concatenated and saved as one row.

As a result, a data frame is formed, for each document, having in each row, the DMC, along with the text, and corresponding applicability ID. The next step is to replace each ID, with the appropriate applicability, found in the applicability data frame saved in the previous step.

C. Resulting Format

The resulting data format of this model is composed of different data frames containing the full lists of the parsed documents. These data frames are saved in different Excel files. Each document is referred to by its unique DMC. Table I shows the different data frames of all parsed documents, and their composition, saved as Excel files.

The data module data frame contains the concatenate DMC, the list of the DMC details, the title, issue number, issue date, type, and the applicability of the entire document. The relationship data frame maps each DMC of a document to the list of referred DMCs in the document. The content data frame lists for each DMC, the text found in its content, grouped by its corresponding applicability. Finally, the preliminary requirements data frame and the close requirements data frame show the requirements of each DM, grouped by their applicabilities as well. For a specific document, all details can be accessed by specifying the DMC. Its main details and content are found in 'Data Module' and 'Content'. If the document refers to others, the list of DMCs can be found in 'Relationships'. If any requirements are needed, they are listed in 'Close Requirements' and 'Preliminary Requirements'. If any specific part is needed in the requirements, its ID leads to the details in 'IPD'. All these details are based on their applicability, that is specified in 'Applicability'.

TABLE I
EXTRACTED DATA FRAMES COMPOSITION

Files	Columns
Applicability	List of all applicabilities extracted
Close Requirements	For each DMC, lists of close requirements grouped by applicability
Content	For each DMC, the full text, grouped by applicability
Data Module	List of DMC, issue number, title, type, applicability.
IPD	List of all catalogue items details.
Preliminary Requirements	For each DMC, lists of preliminary requirements grouped by applicability.
Relationships	list of referenced DMCs in each DMC.



Fig. 3. Example for embedded applicability in nodes- Part 1

This is how the model translates the complex S1000D documents, into simpler, and easier format, while removing unnecessary details, and preserving required critical ones.

V. EXAMPLES AND RESULTS

An example of the content recurrent function, on a 'description' document in French written in the version 2.3 is added. This example shows how special characters were handled, how external reference was replaced, and how the final text was grouped by applicability. In the content section, the 'descript' tag has nine main sections. Each section has different applicability, and multiple children, having also different applicability. The first part of one of these nine main children is detailed. Three applicabilities are listed, with a total of seven main children. Due to confidentiality, part of the text was masked.

Fig. 3 shows a part of the XML document to be parsed. For easier references, applicabilities are respectively named 'A1', 'A2', and 'A3'. The main applicability of the node 'para0' is 'A1', all children of the node should have the same applicability unless they have their own. Fig. 4 shows four other children, where the first has its own applicability 'A3', while the others inherit the main 'A1'. Fig. 5 shows the rest of this node, where an external data module is referenced by


```

<para id="pgfId-12251898">
  <applic>
    <model model="CONF 15">
      <maintlevel>
        <mntlvl mntlvl="m101"/>
      </maintlevel>
    </model>
    <model model="CONF 31">
      <maintlevel>
        <mntlvl mntlvl="m101"/>
      </maintlevel>
    </model>
    <model model="CONF 31 BPC">
      <maintlevel>
        <mntlvl mntlvl="m101"/>
      </maintlevel>
    </model>
  </applic> The [redacted] of
  the equipped [redacted]
  operates out of threat, with
  a [redacted] filter
  and starts automatically
  in ventilation.
</para>
<para id="pgfId-12251649">Setting
filtration system route [redacted] is
carried out by order of the head of
mission, as well as the implementation
in place. </para> A1
<para id="pgfId-12251650">In case of
power failure in the [redacted]
the [redacted] is rescued by the
carrier batteries for approx.
15 minutes. </para> A1
<para id="pgfId-12251654">The [redacted] can
be protected against the [redacted] </para>

```

Fig. 4. Example for embedded applicability in nodes- Part 2

its DMC, thereby should be replaced. The outcome of this node should be, as shown in all figures, a collection of the three applicabilities, where five nodes have 'A1', two having 'A2' and one having 'A3'. The corresponding text should also be complete; references are replaced and special characters are decoded.

Table II shows the outcome of the recurrent content extraction, some words were replaced by XY for confidentiality. The referenced data module was replaced by the appropriate title, and text having same applicability were concatenated together, even for embedded applicabilities, on the finest level. Same results were obtained when tested on different technical documentations. Starting with the bike English data set available online when downloading the S1000D specifications [1]. We also applied it to four other French and English data sets, having different versions of 4.0 and 2.3. Each data set contained between 700 and 20000 documents. Due to client confidentiality, these data sets are not publicly available. Results showed that this model successfully extracted all the data correspondingly to the mentioned rules and logic. Special characters were decoded, especially in the French language, and text was well replaced and extracted based on its applicability.

Note that the extracted text can be then tokenized, lemmatized, or changed into embeddings, based on the machine learning

```

<subparal id="N072F5F38.08C226C0">
  <title> A1
  Setting up the [redacted]
  </title>
  <warning vital="0">
    <applic>
      <model model="CONF 16">
        <maintlevel>
          <mntlvl mntlvl="m101"/>
        </maintlevel>
      </model>
    </applic>
    <para>THIS FILTER IS RELATIVELY HEAVY
    (. 18 KG), IT IS RECOMMENDED
    TO ENSURE THE OPERATOR BY INSTALLING AND
    USING THE [redacted] AND
    THE RETREAT LANYARDS SUPPLIED WITH THE
    GENERAL TOOLS SET
    <refdm>
      <avee>
        <modelic> [redacted] </modelic>
        <sdc>A</sdc>
        <chapnum>60</chapnum>
        <section>6</section>
        <subset>0</subset>
        <subject>00</subject>
        <discode>00</discode>
        <discodev>A</discodev>
        <incode>020</incode>
        <incodev>A</incodev>
        <itemloc>A</itemloc>
      </avee>
      <dmtitle>
        <techname>
          General Tooling [redacted]
        </techname>
        <infoname>Configuration</infoname>
      </dmtitle>
    </refdm>
  </para>
</warning>

```

Fig. 5. Example for data module reference with embedded applicability

technique applied. We also added a part of database insertion and update, but this also can be relatively changed based on the chosen database.

VI. CONCLUSION

In this paper, SALERNO, a python-based software was proposed to convert S1000D-based files to a less complicated format, while preserving the structure's logic. This can be considered as the first step of data preprocessing task to prepare the data for a machine learning model. SALERNO takes as input the file path containing all documents of the technical manual, makes the appropriate conversion and decoding, then filters them based on their types. New and updated files are only parsed, while deleted ones are saved apart. The first step of characteristics extraction saves all documents in a data module data frame, as a row of corresponding DMC, title, type, issue number, and applicability. The second phase extracts the content part of the document. Replaces all internal and external references by their corresponding text, and checks for applicability on all levels. The same logic is applied for preliminary and close requirement extraction, resulting in two data frames respectively grouping the items of each DMC by

TABLE II
EXTRACTED TEXT GROUPED BY APPLICABILITY

Applicability	Text
A1	XY FILTRATION DEVICE. Setting filtration system route XY is carried out by order of the head of mission, as well as the implementation in place. In case of power failure in the XY the XY is rescued by the carrier batteries for approx. 15 minutes. The XY can be protected against the XY. Setting up the XY.
A2	The XY arranged works out of threat, with a filter XY and started automatically in ventilation. THIS FILTER IS RELATIVELY HEAVY (. 18 KG), IT IS RECOMMENDED TO ENSURE THE OPERATOR BY INSTALLING AND USING THE XY AND THE RETREAT LANYARDS SUPPLIED WITH THE GENERAL TOOLS SET General Tooling XY.
A3	The XY the equipped XY operates out of threat, with a XY filter and starts automatically in ventilation.

their applicability. The model was applied to different data sets, of two different versions, in both English and French languages. Results showed that all issues were covered, even for the most complicated documents. In future work, authors will apply the appropriate method to change the resulting text into embeddings, using them in a machine learning model that helps the search of certain document in huge S1000D technical manuals. But this is not limited for so, the efficient result of this model have enough context to be used as alternative of complex XML documents for any other machine learning or data mining models.

REFERENCES

- [1] "S1000d." [Online]. Available: <https://s1000d.org/>
- [2] "S-series of ils specifications." [Online]. Available: <https://www.asd-europe.org/s-series-ils-specifications>
- [3] Y. Ma, X. Cheng, M. Zhang, L. Meng, B. Luo, and Z. Cheng, "Data conversion method based on prior knowledge template updating," in *2019 Chinese Automation Congress (CAC)*. IEEE, 2019, pp. 140–145.
- [4] "Scorm." [Online]. Available: <https://scorm.com/>
- [5] X. Ma, X. Wang, X. Zhao, Z. Li, and R. Jia, "Research on ietm authoring platform architecture based on s1000d specification," in *2015 12th IEEE International Conference on Electronic Measurement & Instruments (ICEMI)*, vol. 1. IEEE, 2015, pp. 497–501.
- [6] S. Qi and W. Zhou, "Design and implementation of process data module editor based on s1000d," in *World Automation Congress 2012*. IEEE, 2012, pp. 1–4.
- [7] Z. Weijiao, S. Peng, and C. Yan, "S1000d based standardized technical information management for high-speed trains," in *2015 Fifth International Conference on Instrumentation and Measurement, Computer, Communication and Control (IMCCC)*. IEEE, 2015, pp. 987–991.
- [8] "Aerospace and defence industries association of europe (asd)." [Online]. Available: <https://www.asd-europe.org/>
- [9] "Aerospace industries association of america (aia)." [Online]. Available: <https://www.aia-aerospace.org/>
- [10] "Ata e-business program." [Online]. Available: <https://ataebiz.org/>